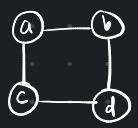
॥ श्रीगणेशाय नमः ॥

Graphs

Graphs are sets of nodes and edges.

Representation of graphs

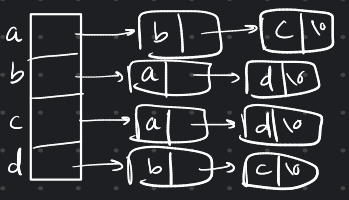


Adjacency matrix: → use for dense graphs

	a	b	c	d
a	0	1	1	0
b	1	0	0	1
c	1	0	0	1
d	0	1	1	0

space req : $O(v*v) = O(v^2)$

Adjacency list: → use for sparse graphs



2e edges

space $= O(V+E)$ (for an undirected graph)

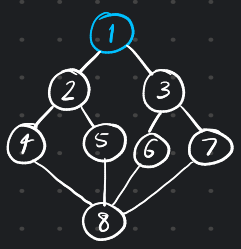
visited: we've seen that node

explored: we've seen a node and all its neighbors.

we keep all the unexplored nodes in a data structure (fringe).

Breadth-first search

```
BREADTH_FIRST(v):
    -- Graph G, arr visited[]
    -- visited[] is initially all false
    u = v, visited[v] = true
    loop:
        for all vertices w in G adjacent to u do
            if visited[w] == false then
                enqueue(w)
                visited[w] = true
            if queue is empty then
                break
            u = dequeue()
```



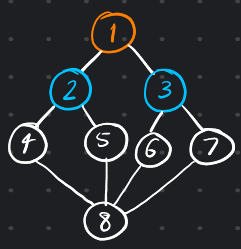
	1	2	3	4	5	6	7	8
visited	1	0	0	0	0	0	0	0

→ O(V) to initialize

queue

u = 1

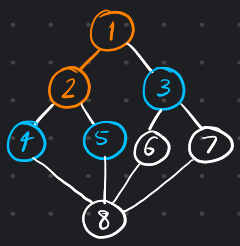
adj. vertices = {2, 3}



	1	2	3	4	5	6	7	8
visited	1	1	1	0	0	0	0	0

→ O(V) to initialize

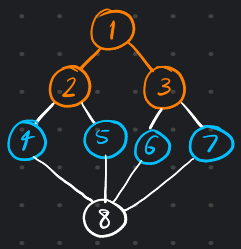
queue: 2 | 3



	1	2	3	4	5	6	7	8
visited	1	1	1	1	1	0	0	0

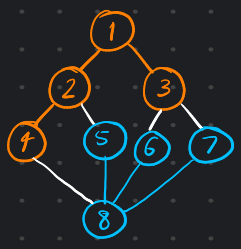
→ O(V) to initialize

queue: ~~2~~ | 3 | 4 | 5



visited: 1 2 3 4 5 6 7 8 — 1 0 0 0 0 0 0 0 — O(V) to initialize

queue: ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ 6 7



visited: 1 2 3 4 5 6 7 8 — 1 0 0 0 0 0 0 0 — O(V) to initialize

queue: ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ 6 7 8



visited: 1 2 3 4 5 6 7 8 — 1 0 0 0 0 0 0 0 — O(V) to initialize

queue: ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ ~~7~~ ~~8~~

Space:
- Visited: O(n)
- Queue: O(n)

Time:
- To initialize visited[]: O(V)
- To visit all adj lists: O(E)
- Total
- = O(V+E)

- To visit all elements in matrix: $O(V^2)$
- Total
- = $O(V^2 + V)$
- = $O(V^2)$

If adj list is used:



all these need to be explored.

we'll visit one list, then not touch it again

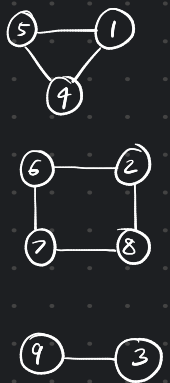
O(2e) elements in list

If adj. matrix is used,
one row is visited for a node.



$O(V^2)$ total

A search only checks reachable nodes out. A graph may be disconnected. To perform a traversal, we may need to perform ultiple searches.

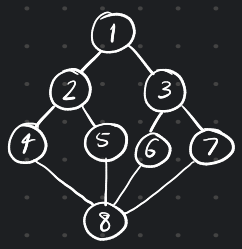


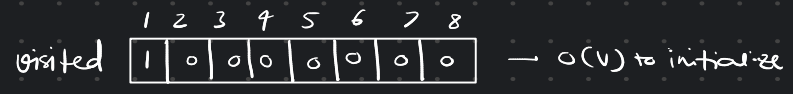
BFS(1)
BFS(2)
BFS(3)
} → BFT(G)

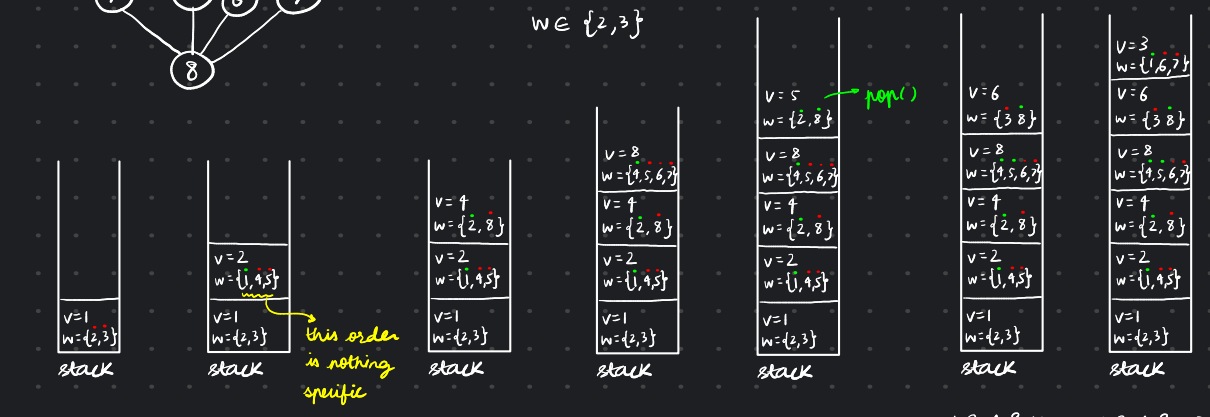
```
BREADTH_FIRST_TRAVERSAL(G, n):
    for i = 1 to n do
        visited[i] = false
    for i = 1 to n do
        if visited[i] == false then
            BREADTH_FIRST(i)
```

Depth-first search

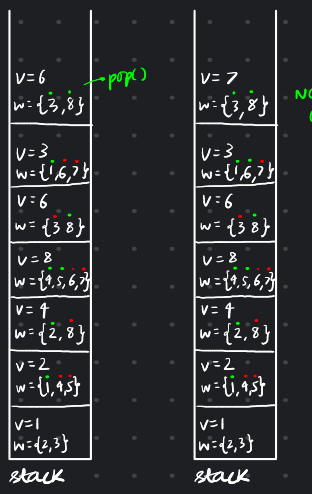
```
DEPTH_FIRST(v):
    visited[v] = true
    for each vertex w adjacent to v do
        if visited[w] == false then
            DEPTH_FIRST(w)
```







visited 1 | 1,2 | 1,2,4 | 1,2,4,8 | 1,2,4,8 | 1,2,4,8,5 | 1,2,4,8,5,3



1,2,4,8,5,3,6 | 1,2,4,8,5,3,6,7

Space: O(n) (for visited + stack both)

Time:

- O(V) to initialize visited
  + O(E) for adj list

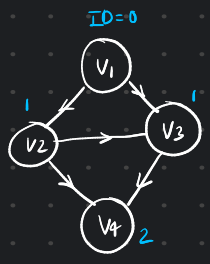
- O(V) to init visited
  + $O(V^2)$ for adj matrix

Time = $O(V+E)$
OR
$O(V^2)$

```
DEPTH_FIRST_TRAVERSAL(G, n):
    for i = 1 to n do
        visited[i] = false
    for i = 1 to n do
        if visited[i] == false then
            DEPTH_FIRST(i)
```

Directed acyclic graph

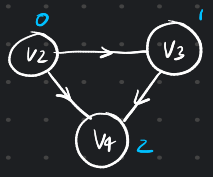
Topological sort on DAG



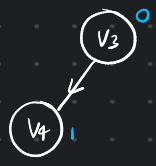
We start with nodes with indegree = 0.

V1

We delete it and its outgoing edges.



V2 next-

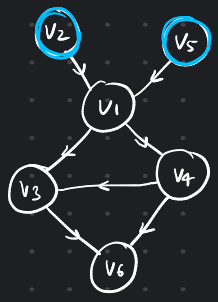


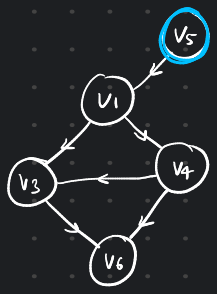
V3

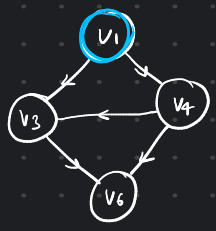


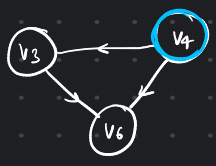
V4

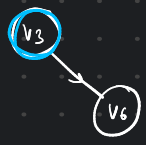
order: V_1, V_2, V_3, V_4 (not necessarily unique)













order: $V_2\ V_5\ V_1\ V_4\ V_3\ V_6$
OR
$V_5\ V_2\ V_1\ V_4\ V_3\ V_6$

Step 1: Identify vertices that have no incoming edges. ← $O(V^2)$ (V 1 time) → can be reduced: whenever edge is deleted, put nodes of indegree 0 into a queue. Then picking a node from that would be O(1). So just the initial O(V) would be needed.

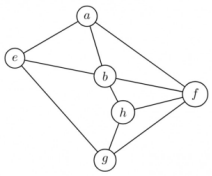
Step 2: Delete a vertex with indegree 0 and all its outgoing edges from the graph. ← $O(E)$

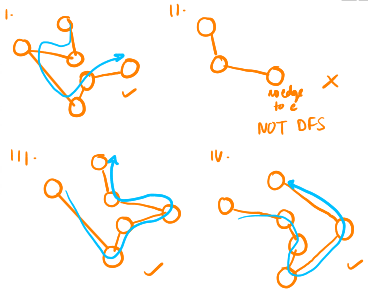
Repeat steps 1 and 2 till the graph is empty.

$T = O(V+E)$

1.9.5 Graph Search: GATE CSE 2003 | Question: 21

Consider the following graph:





Among the following sequences:

I. abeghf
II. abfehg
III. abfhge
IV. afghbe

Which are the depth-first traversals of the above graph?

A. I, II and IV only B. I and IV only C. II, III and IV only D. I, III and IV only

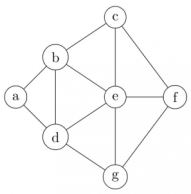
gatecse-2003 algorithms graph-algorithms normal graph-search

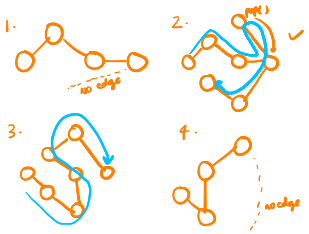
1.9.22 Graph Search: GATE IT 2008 | Question: 47

Consider the following sequence of nodes for the undirected graph given below:

1. $a\ b\ e\ f\ d\ g\ c$
2. $a\ b\ e\ f\ c\ g\ d$
3. $a\ d\ g\ e\ b\ c\ f$
4. $a\ d\ b\ c\ g\ e\ f$

A Depth First Search (DFS) is started at node a. The nodes are listed in the order they are first visited. Which of the above is/are possible output(s)?



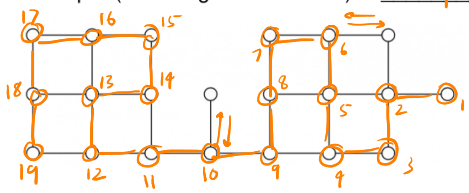


A. 1 and 3 only B. 2 and 3 only C. 2,3 and 4 only D. 1,2 and 3 only

gateit-2008 algorithms graph-algorithms normal graph-search depth-first-search

1.9.10 Graph Search: GATE CSE 2014 Set 3 | Question: 13

Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is ____19____.

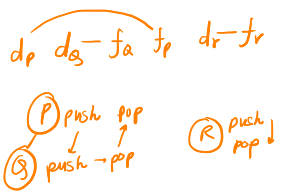


gatecse-2014-set3 algorithms graph-algorithms numerical-answers normal graph-search

1.9.20 Graph Search: GATE IT 2006 | Question: 47

Consider the depth-first-search of an undirected graph with 3 vertices P, Q, and R. Let discovery time $d(u)$ represent the time instant when the vertex u is first visited, and finish time $f(u)$ represent the time instant when the vertex u is last visited. Given that

$d(P) = 5$ units	$f(P) = 12$ units
$d(Q) = 6$ units	$f(Q) = 10$ units
$d(R) = 14$ unit	$f(R) = 18$ units



Which one of the following statements is TRUE about the graph?

A. There is only one connected component
B. There are two connected components, and P and R are connected
C. There are two connected components, and Q and R are connected
D. There are two connected components, and P and Q are connected

gateit-2006 algorithms graph-algorithms normal graph-search depth-first-search

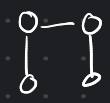
Types of graphs



max # of edges for n nodes = $^nC_2 = O(n^2)$

so for a complete graph, K_n, $E = O(V^2)$ or $V = O(\text{sqrt } E)$

Q. For 4 nodes, how many edges are reqd to connect them?



n nodes → min. (n-1) edges to connect

If we connect a n vertex graph with n-1 edges, it is called a spanning tree.

K_3:



of spanning trees:



3

K_4 (complete)

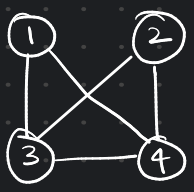


and so on... $4^{4-2} = 16$

of spanning trees for K_n (complete graph with n vertices) $= n^{n-2}$

Kirchoff's theorem

For a non-complete graph, (or any) construct the adjacency matrix.



	1	2	3	4
1	0	0	1	1
2	0	0	1	1
3	1	1	0	1
4	1	1	1	0

−1

replace with −1

replace with degree

Diagonal 0's → degree
Non-diagonal 1's → −1
Non-diagonal 0's → 0's

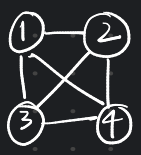
$$\begin{bmatrix} 2 & 0 & 1 & 1 \\ 0 & 2 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}$$

→ cofactor of any element = # of spanning trees

$2(9-1) - (-1)(-3-1) - 1(1+3)$

$= 16 - 4 - 4 = 8$

The graph has 8 spanning trees.

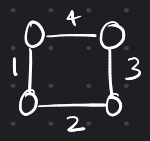


$$\begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}$$

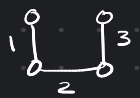
$3(9-1)+1(-3-1)-1(1+3)$

$24-4-4=16$

↳ $4^{4-2}=16$



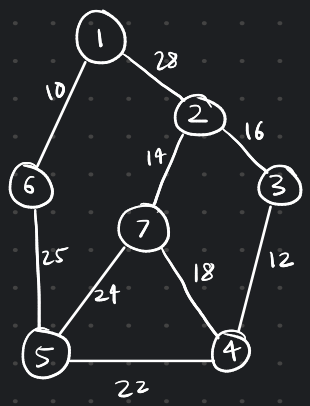
→

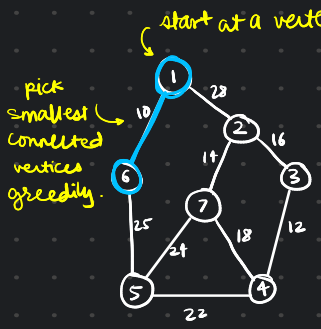


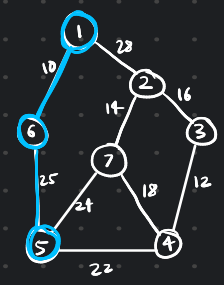
min. cost spanning tree.

To find the minimum cost spanning tree, we can use:
- Prim's algorithm
- Kruskal's algorithm

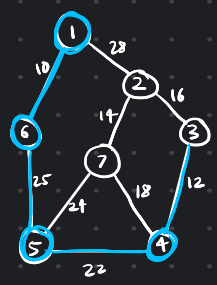
Prim's algorithm

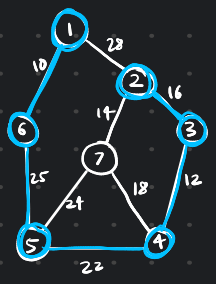


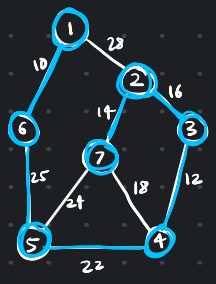


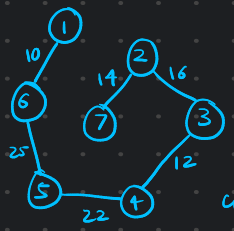


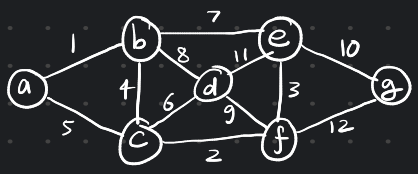


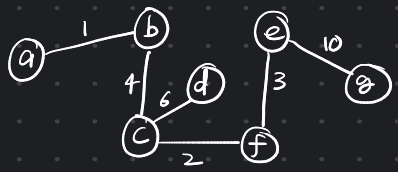










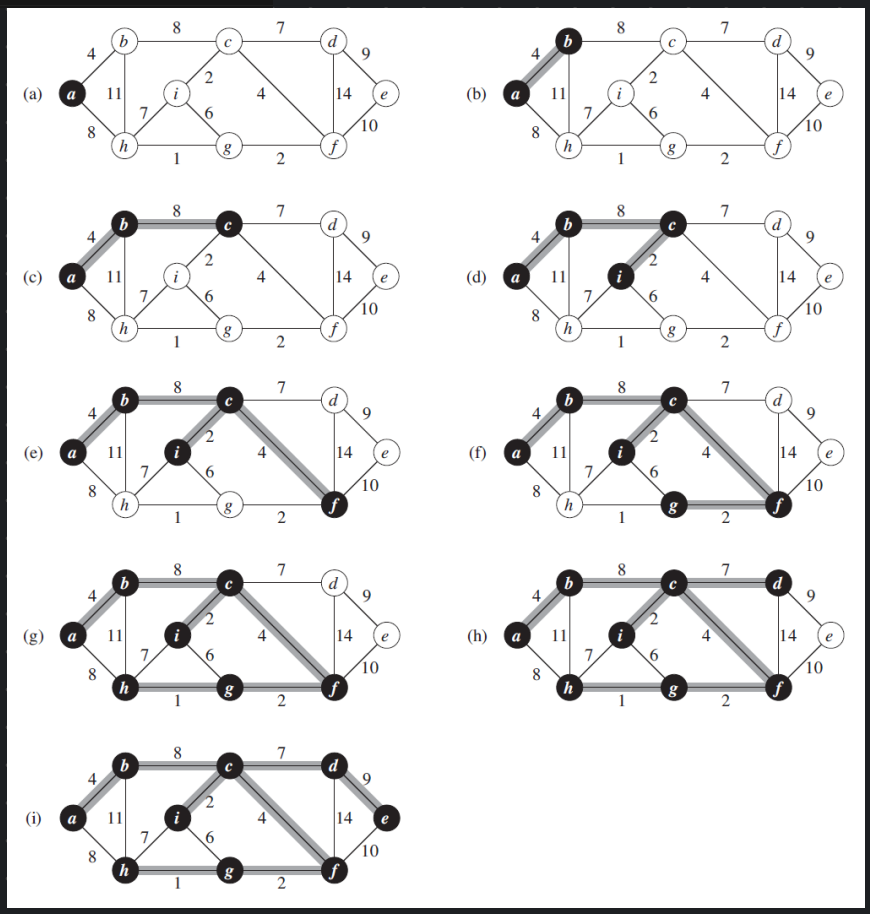


cost = 26

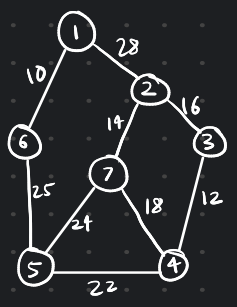
Is the MST unique? → If the edge weights are unique, then yes. Else that may not be the case. There could be many MSTs when edges are repeated.

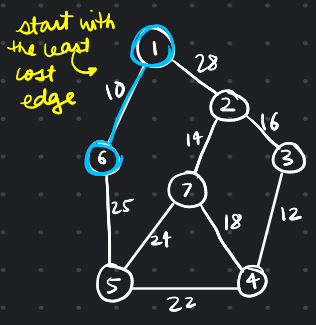
Minimum edge weight data structure	Time complexity (total)
adjacency matrix, searching	$O(\|V\|^2)$
binary heap and adjacency list	$O((\|V\| + \|E\|) \log \|V\|) = O(\|E\| \log \|V\|)$
Fibonacci heap and adjacency list	$O(\|E\| + \|V\| \log \|V\|)$

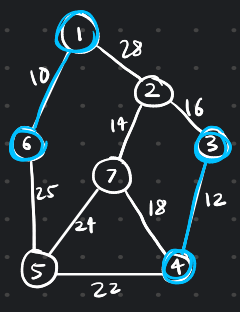
```
MST-PRIM(G, w, r)
 1  for each u ∈ G.V
 2      u.key = ∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = G.V
 6  while Q ≠ ∅
 7      u = EXTRACT-MIN(Q)
 8      for each v ∈ G.Adj[u]
 9          if v ∈ Q and w(u, v) < v.key
10              v.π = u
11              v.key = w(u, v)
```



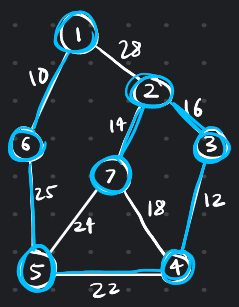
Kruskal's algorithm







pick up edges greedily



cost = 99

```
                MST-KRUSKAL(G, w)
O(1)    1   A = ∅
O(V)    2   for each vertex v ∈ G.V
        3       MAKE-SET(v)
O(E logE) 4 sort the edges of G.E into nondecreasing order by weight w
        5   for each edge (u, v) ∈ G.E, taken in nondecreasing order by weight
        6       if FIND-SET(u) ≠ FIND-SET(v)
O(V logV) 7         A = A ∪ {(u, v)}
        8           UNION(u, v)
        9   return A
```

1.18.6 Minimum Spanning Tree: GATE CSE 2000 | Question: 2.18

Let G be an undirected connected graph with distinct edge weights. Let e_{max} be the edge with maximum weight and e_{min} the edge with minimum weight. Which of the following statements is false?

A. Every minimum spanning tree of G must contain e_{min} ← first step in kruskal's
B. If e_{max} is in a minimum spanning tree, then its removal must disconnect G ← we'll add e max only when it is the only edge connecting any 2 nodes, so yes.
C. No minimum spanning tree contains e_{max} ← False (case in opt. b)
D. G has a unique minimum spanning tree ← true since edge weights are distinct.

gatecse-2000 algorithms minimum-spanning-tree normal

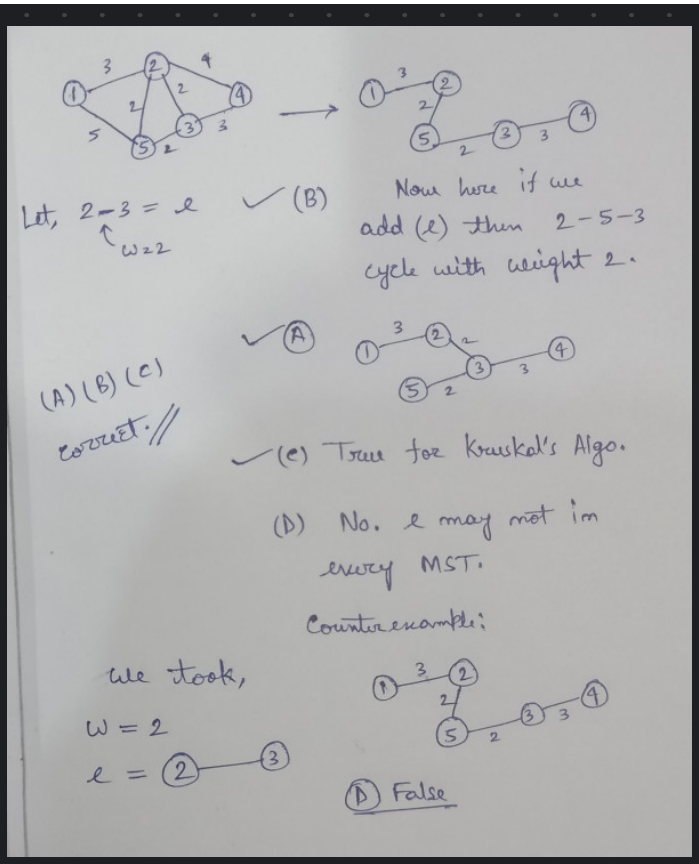
1.18.12 Minimum Spanning Tree: GATE CSE 2007 | Question: 49

Let w be the minimum weight among all edge weights in an undirected connected graph. Let e be a specific edge of weight w. Which of the following is FALSE?

A. There is a minimum spanning tree containing e ← there'd be atleast 1 such tree, yes.
B. If e is not in a minimum spanning tree T, then in the cycle formed by adding e to T, all edges have the same weight.
C. Every minimum spanning tree has an edge of weight w ← By kruskal's we'd have to pick it at the first edge.
D. e is present in every minimum spanning tree ← Not necessarily, the edge weights are not distinct.

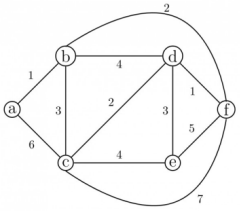
FALSE. ANS = D

If e is not part of a minimum spanning tree, then all edges which are part of a cycle with e, must have weight $\leq e$, as otherwise we can interchange that edge with e and get another minimum spanning tree of lower weight. So, **B** and **A** are also TRUE.



1.18.11 Minimum Spanning Tree: GATE CSE 2006 | Question: 47

Consider the following graph:



Which one of the following cannot be the sequence of edges added, **in that order**, to a minimum spanning tree using Kruskal's algorithm?

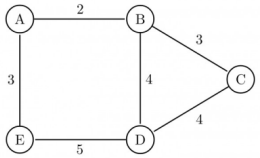
A. $(a-b),(d-f),(b-f),(d-c),(d-e)$
B. $(a-b),(d-f),(d-c),(b-f),(d-e)$
C. $(d-f),(a-b),(d-c),(b-f),(d-e)$
D. $(d-f),(a-b),(b-f),(d-e),(d-c)$

gatecse-2006 algorithms graph-algorithms minimum-spanning-tree normal

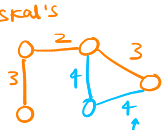
1.18.3 Minimum Spanning Tree: GATE CSE 1995 | Question: 22

How many minimum spanning trees does the following graph have? Draw them. (Weights are assigned to edges).

2



Kruskal's



pick either of these

gate1995 algorithms graph-algorithms minimum-spanning-tree easy descriptive

1.18.4 Minimum Spanning Tree: GATE CSE 1996 | Question: 16

A complete, undirected, weighted graph G is given on the vertex $\{0, 1, \ldots, n-1\}$ for any fixed 'n'. Draw the minimum spanning tree of G if

A. the weight of the edge (u, v) is $|u - v|$

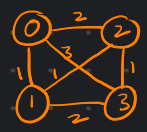


B. the weight of the edge (u, v) is $u + v$



gate1996 algorithms graph-algorithms minimum-spanning-tree normal descriptive

A.

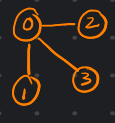




line graph.

B.





star.

Dijkstra's algorithm

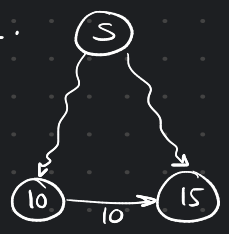
Relaxation: If $d(u) > d(v) + c(v,u)$ $\Rightarrow$ $d(u) = d(v) + c(u,v)$

e.g.

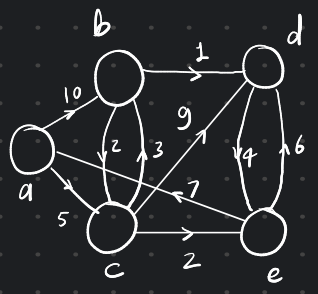


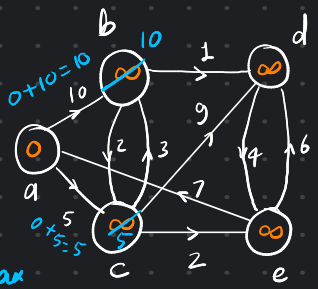
here we'll relax to get a better distance to u.

ex. 2.

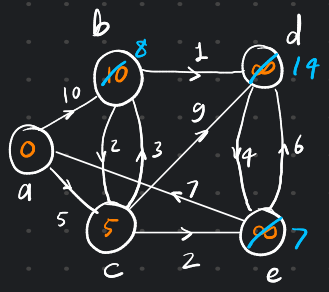


here relaxation is useless as $10 + 10 = 20$ worse than 15.

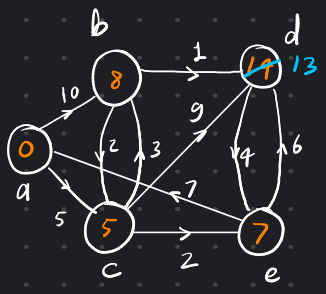




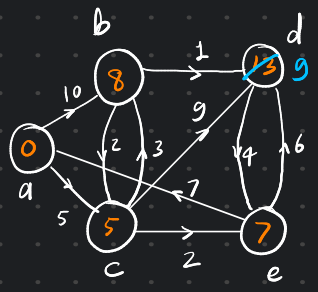
relax outgoing edges from a



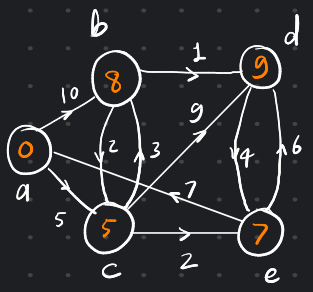
choose smallest and relax (relax for c)



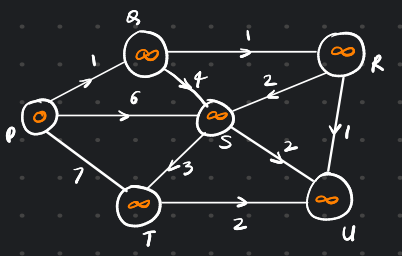
of b, d, e
10 14 7
relax e
e to a → No relaxa^ $7+7=14$ $0 \neq 14$
e to d → $7+6=13 < 14$ relax

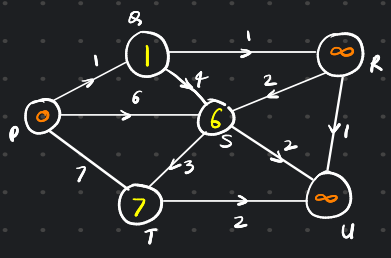


b d
8 13
↑ relax
b-d $8+1=9$ $13>9$
relax



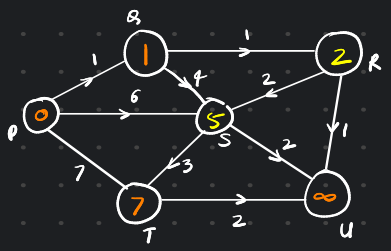
All relaxa^s are done.





P → Q 1
P → T 7
P → S 6

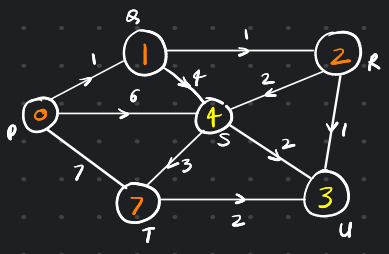
P



Q S T
1 6 7

Q → R ✓
Q → S 5 ✓

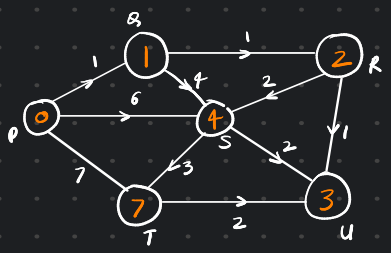
Q



R S T
2 5 7

R → U 3 ✓
R → S 4 ✓

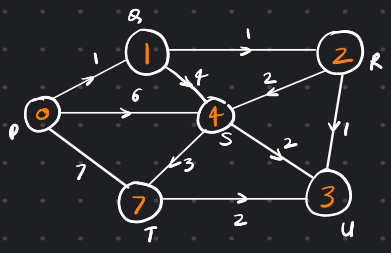
R



S T U
4 7 3
,

no outgoing edges

U



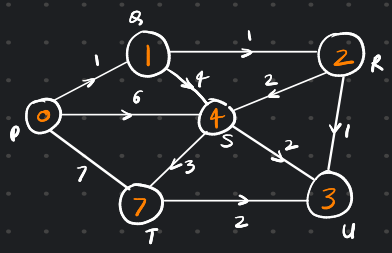
S T
4 7
,

S → T 4+3=7 same. no update

S → U
↳ already done

No relaxn's

S



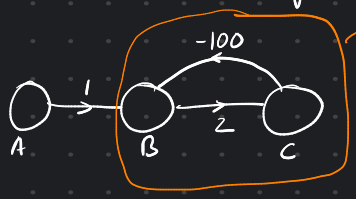
T

order : PQRUST

NOTE: once you've relax for a vertex, its value is not changed later.

Dijkstra's algorithm doesn't work if there are any negative cycles present.

Example where Dijkstra's fails:



Dijkstra's doesn't detect -ve cycles.
So it doesn't allow -ve edge weights.

Bellman-Ford algo can detect -ve cycles.

```
DIJKSTRA(Graph G, Weights W, Source S):
    Initialize single-source G, S   ← O(V)
    S = ϕ   -- S will have nodes for which the shortest path is found out
    Q = G.V     -- min heap

    while Q != ϕ:
        u = extract_min(Q)
        S = S union {u}
        for each vertex v in G.adj(u):
            relax(u, v, w)  -- for heap, relax means decrease_key
                                                    ↳ O(E log V)
```

Time complexity : $O(E \log V)$

Bellman ford algorithm

Unlike Dijkstra's, the Bellman-Ford algorithm can detect negative cycles.
It works on the observation that the shortest path will have atmost n-1 edges (where n = # of vertices).

1st pass: find the shortest path containing atmost 1 edge.
2nd pass: find the shortest path containing atmost 2 edges.
⋮
(n-1)th pass: find the shortest path containing atmost n-1 edges. So V-1 passes.

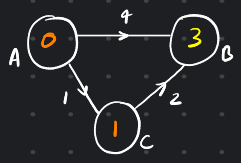
1 additional pass: to detect -ve weight cycles (After v-1 passes, all shortest paths are found. If we still get a shorter path in one more pass, then there's a -ve weight cycle).

ALGORITHM
(V-1)E relaxations. → in any order (no min heap needed). so each relaxation would take O(1).

$T = O(V * E)$

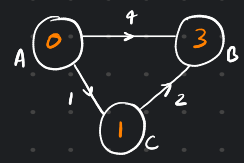


(n-1) passes to find the shortest paths.



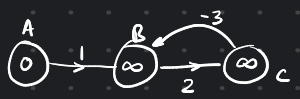
---

1 pass to check for -ve cycles.



No weight change.

3rd pass









relaxing in the 3rd pass changes distances.
⇒ there's a -ve cycle.

```
BELLMAN_FORD(Graph G, Weights W, Source S):
    Initialize single-source G, S
    for i = 1 to V-1:
        for each edge [u, v] in E:
            relax(u, v, w)

    for each edge [u, v] in E:
        if v.d > u.d + w(u, v):
            return False

    return True
```

→ (V-1)E = O(V*E)

→ O(E)

$T = O(VE)$